# Episode 01 — Helm vs Kubectl: Why Templates Matter

## Season 1 · Helm Foundations

---

## The Hook

You have three environments: dev, staging, production. Your Kubernetes app has a Deployment, a Service, a ConfigMap, an Ingress, and a HorizontalPodAutoscaler. That's 5 YAML files. Across 3 environments that's 15 files — all nearly identical. When you add a new environment variable, you update 15 files. When you rename a label, you hunt through 15 files. When you want to deploy version v2.1.0 only to staging, you manually edit one of those 15 files and hope you picked the right one.

There's a better way. It's called Helm, and it turns those 15 files into 5 templates + 3 values files. One change, everywhere.

---

## Production Scenario

**Setup**: A platform team manages 12 microservices × 4 environments = 48 Kubernetes Deployment YAML files. Each service has 6 YAML files. That's 288 files in total. A security team mandates adding a new `securityContext` to every container across all deployments. One engineer spends 3 days updating every file, introduces inconsistencies across environments, and misses 4 files that are found in a security audit 2 weeks later.

**With Helm**: The `securityContext` is added to the Deployment template once. All 12 services across all 4 environments pick it up on the next `helm upgrade`. The change takes 20 minutes.

---

## Core Concepts

### The YAML Sprawl Problem

Without Helm, a real production microservice deployment looks like:

```
my-service/
├── deployment-dev.yaml
├── deployment-staging.yaml
├── deployment-prod.yaml
├── service-dev.yaml
├── service-staging.yaml
├── service-prod.yaml
├── configmap-dev.yaml
├── configmap-staging.yaml
├── configmap-prod.yaml
├── ingress-dev.yaml
├── ingress-staging.yaml
├── ingress-prod.yaml
```

```
└── hpa-prod.yaml           # Only prod has HPA
```

**Problems**:

- Any cross-cutting change (new label, security context, annotation) requires touching every file
- Environment differences are hidden in diffs across files instead of explicit
- No versioning — you don't know what was deployed when
- No rollback mechanism built in
- Sharing the service with another team requires copying all these files

## What Helm Solves

Helm solves three core problems:

```
Problem 1: Configuration Repetition
  Before: 12 near-identical YAML files
  After:  1 template + values overrides per environment

Problem 2: No Release Management
  Before: kubectl apply → no history, no rollback
  After:  helm install → versioned release history, one-command rollback

Problem 3: No Distribution Mechanism
  Before: email/Slack YAML files to other teams
  After:  publish to chart registry, install with helm install
```

## Helm vs Alternatives

| Tool | Approach | Best For |
|---|---|---|
| **kubectl** | Imperative/declarative YAML | Quick fixes, simple resources |
| **Helm** | Go templates + values | Reusable, distributable packages |
| **Kustomize** | Overlay patches | Owning base + patches without templates |
| **jsonnet** | Functional config language | Complex config generation |
| **Pulumi/CDK8s** | Real programming languages | Infrastructure as actual code |

**When to use Helm**:

- You distribute your app to external teams or customers
- You need parameterized configuration (dev/staging/prod values)
- You want release lifecycle management (history, rollback)
- You use third-party apps that ship as Helm charts (Prometheus, nginx-ingress, cert-manager)

**When Kustomize might be better**:

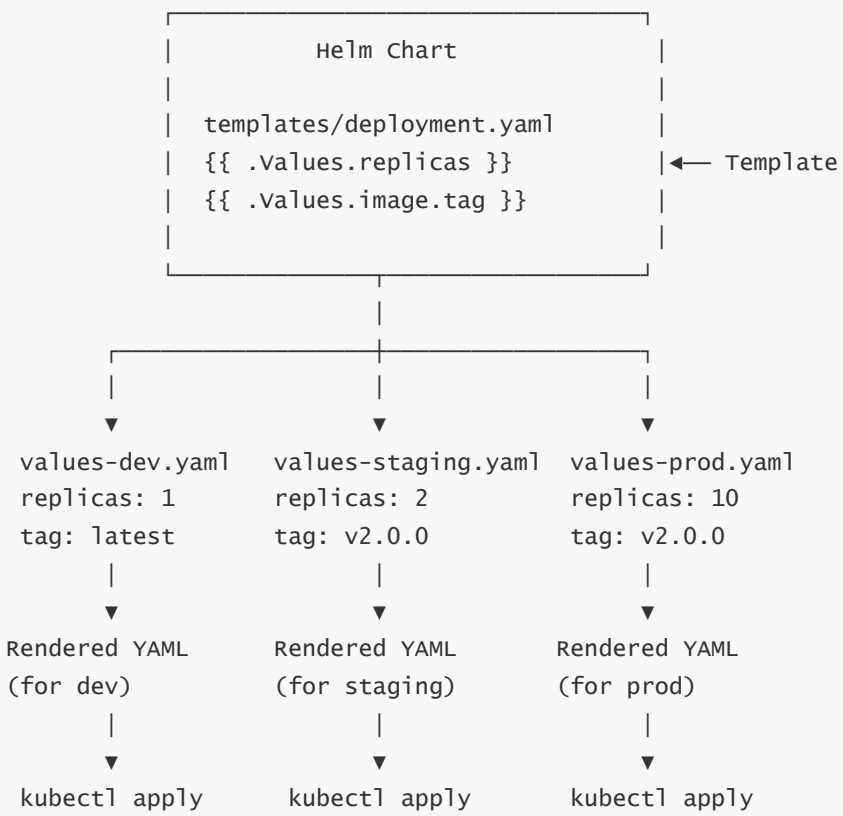
- You own all the YAML and don't need templating

- You want plain YAML in Git (auditable, no template rendering)
- You're adding small patches on top of base manifests

**In practice**: most teams use both — Helm for third-party apps, Kustomize for their own services.

---

# The Helm Mental Model




---

# Lab — Feel the Difference

**Objective**: Deploy the same application with kubectl 3 times (different configs), then do it with Helm once.

## Part A: The kubectl way (painful)

```
# Create dev deployment manually
kubectl apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-dev
  namespace: dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
      env: dev
```

```
  template:
    metadata:
      labels:
        app: myapp
        env: dev
    spec:
      containers:
        - name: myapp
          image: nginx:1.25
          env:
            - name: ENV
              value: "development"
          resources:
            requests: { memory: "64Mi", cpu: "50m" }
            limits:   { memory: "128Mi", cpu: "100m" }
EOF

# Repeat for staging (copy-paste, change env name, replicas, resources)
# Repeat for production (copy-paste again)
# Now add an annotation to ALL THREE... (painful)
```

## Part B: The Helm way

```
# Install Helm
brew install helm

# Create chart
helm create myapp

# Install to dev
helm install myapp-dev ./myapp \
  --set replicaCount=1 \
  --set image.tag=1.25 \
  --namespace dev --create-namespace

# Install to staging
helm install myapp-staging ./myapp \
  --set replicaCount=2 \
  --set image.tag=1.25 \
  --namespace staging --create-namespace

# Install to prod
helm install myapp-prod ./myapp \
  --set replicaCount=10 \
  --set image.tag=1.25 \
  --namespace prod --create-namespace

# List all releases
helm list --all-namespaces

# Add annotation everywhere — ONE template change, upgrade all
helm upgrade myapp-dev ./myapp --reuse-values
```

```
helm upgrade myapp-staging ./myapp --reuse-values
helm upgrade myapp-prod ./myapp --reuse-values

# Rollback prod to previous version instantly
helm rollback myapp-prod 1
```

---

## Troubleshooting

### Issue 1: `helm: command not found`

```
# Mac: install via Homebrew
brew install helm

# Linux: install via script
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Verify
helm version
# version.BuildInfo{Version:"v3.x.x", ...}
```

### Issue 2: `Error: INSTALLATION FAILED: cannot re-use a name that is still in use`

```
# A release with this name already exists
helm list --all-namespaces | grep my-release

# Uninstall the existing one first
helm uninstall my-release -n my-namespace

# Or use --upgrade flag
helm upgrade --install my-release ./chart
```

### Issue 3: `Error: Kubernetes cluster unreachable`

```
# Helm uses your kubeconfig
kubectl cluster-info          # Is kubectl working?
echo $KUBECONFIG              # Check config path
kubectl config current-context

# Set correct context
kubectl config use-context my-cluster
```

---

# Enterprise Notes

> **GitOps platforms rely on Helm**: ArgoCD and Flux both have native Helm support. When you define an ArgoCD Application pointing at a Helm chart repo + values file, ArgoCD runs `helm template` under the hood and applies the result. Understanding Helm is essential for working with any GitOps platform.

> **Most SaaS platforms ship as Helm charts**: Prometheus, Grafana, nginx-ingress, cert-manager, ArgoCD itself, Vault, Jaeger, Kafka — all distributed as Helm charts on Artifact Hub. Knowing Helm means you can install and configure the entire cloud-native ecosystem without writing custom YAML.

> **The "why not just use kubectl" interview answer**: kubectl is a deployment tool, not a package manager. Helm adds: (1) parameterization — the same chart works in dev and prod with different values; (2) release lifecycle — every install creates a versioned release with rollback capability; (3) distribution — charts can be published and shared like npm packages. For anything beyond a single throwaway deployment, Helm wins.

---

# Interview Prep

**Q: Why isn't kubectl apply enough for managing Kubernetes applications in production?**

A: `kubectl apply` deploys resources but provides no higher-level abstractions. It has no concept of a "release" — you can't see what was deployed at what time, you can't rollback easily, and there's no built-in way to parameterize manifests for different environments. With 10+ services across multiple environments, maintaining hundreds of nearly-identical YAML files becomes unmaintainable. Helm adds templating (one chart works across environments), release management (versioned history, one-command rollback), and distribution (charts shareable via registries), all of which are essential at production scale.

---

**Q: When would you choose Kustomize over Helm?**

A: Kustomize is better when you own all your manifests and want to avoid the indirection of a template language. Kustomize keeps everything as valid Kubernetes YAML — you can read any file and know exactly what it will create. There's no `{{ }}` syntax to decode. It's particularly good for: adding small patches to base manifests, teams that want full transparency without a rendering step, and GitOps setups where every committed file is exactly what runs in the cluster. Helm is better when you're distributing an application to users or teams who shouldn't have to understand the underlying YAML — they just provide values.

---

**Q: What are the three main things Helm provides that kubectl doesn't?**

A: (1) **Templating**: Helm charts use Go templates to parameterize Kubernetes YAML, so one chart can render differently for dev vs prod using different `values.yaml` files. (2) **Release management**: every `helm install` creates a tracked release stored as a Secret in Kubernetes. You get version history, the ability to see what exact values were used, and one-command rollback with `helm rollback`. (3) **Distribution**: charts can be packaged and published to Helm repositories (like Artifact Hub), allowing anyone to install your application with `helm install myapp myrepo/myapp` — like pip or npm for Kubernetes.

---

**Q: How does Helm store its release state?**

A: Helm stores release state as Kubernetes Secrets in the same namespace as the release (by default). Each upgrade creates a new Secret with the rendered manifests, values used, and metadata. This is how Helm tracks history and enables rollback — it simply re-applies a previous Secret's rendered manifests. You can see these Secrets with `kubectl get secrets -l owner=helm`. This is why Helm doesn't need a central server (unlike Helm 2 which had Tiller) — the cluster itself is the state store.

---

## Cheat Sheet

```
# Install Helm
brew install helm                  # macOS
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Verify
helm version

# Add repositories
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

# Search charts
helm search repo nginx
helm search hub wordpress          # Search Artifact Hub

# Install/upgrade
helm install <release> <chart>
helm upgrade <release> <chart>
helm upgrade --install <release> <chart>   # Install if not exists, upgrade if exists

# List and manage
helm list -A                       # All namespaces
helm status <release>
helm history <release>
helm rollback <release> <revision>
helm uninstall <release>

# Inspect
helm show values <chart>           # Default values
helm show chart <chart>            # Chart metadata
helm get values <release>          # Values used for this release
helm get manifest <release>        # Rendered YAML that was applied
```

---

## What's Next

Episode 02 covers **Helm Architecture** — the complete picture of how Charts, Values, and Releases fit together. You'll explore a real Helm chart's directory structure, understand how the template rendering pipeline works, and learn how Helm tracks your deployment history so you can rollback in seconds.